

Sales Copilot — App PRD

Real-time AI meeting assistant · Desktop app · Built for your portfolio

1. Overview

A desktop window that listens to your meeting, transcribes in real time, and shows AI coaching suggestions — without joining the call or appearing to any participant.

INPUTS	OUTPUTS
• Microphone audio (your voice) • System speaker audio (their voice) • Uploaded sales documents (PDF/TXT) • ICP, playbooks, vocabulary guides	• Live rolling transcript • Questions to ask next • Objection responses • Relevant talking points

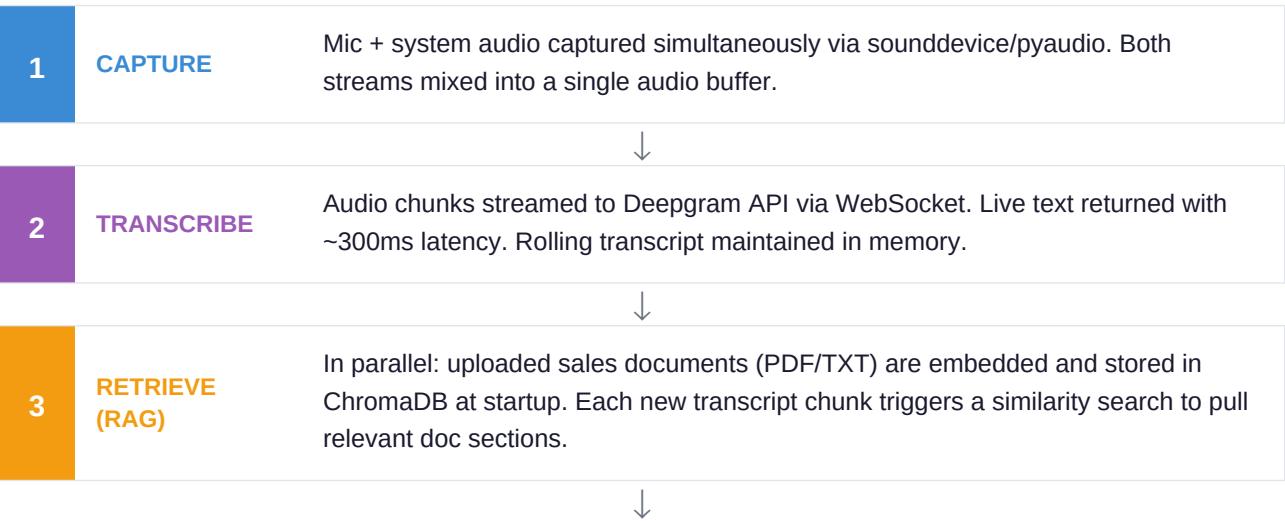
Tech Stack (free tier friendly)

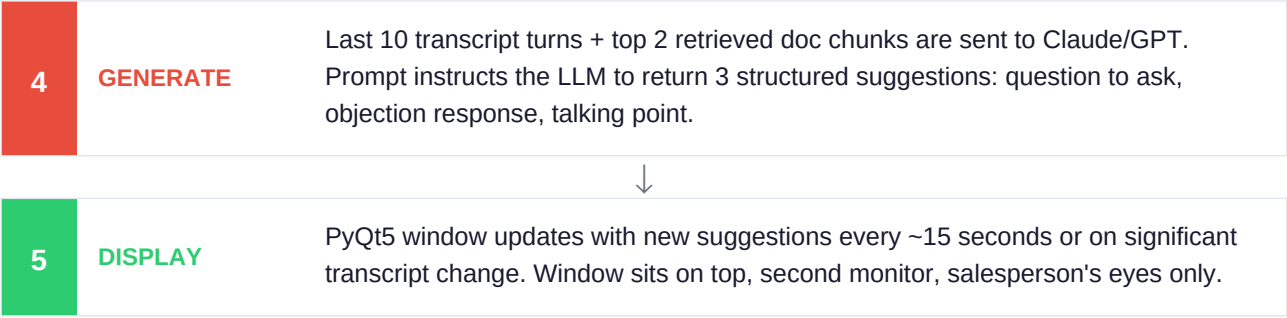
Audio capture	Record mic + system audio locally	sounddevice / pyaudio	Free
Transcription	Stream audio → text, low latency	Deepgram API	\$200 free credit
Document store (RAG)	Embed + retrieve sales docs	ChromaDB	Free, local
LLM suggestions	Generate coaching tips from context	Claude / OpenAI API	Free credits
Desktop UI	Window on second monitor	PyQt5	Free
Packaging	Export as .exe / .app	PyInstaller	Free

Estimated real cost to build and demo: ~\$0–5 USD using free tiers

2. End-to-End Architecture Flow

Follow audio from your microphone all the way to the suggestion shown on screen. Each layer is a distinct processing step.





3. Desktop Window Layout

The app is a narrow vertical window (~320×600px) designed to sit on a second monitor, always on top, invisible to meeting participants.

Title bar	App name + live recording indicator (● REC)	Always visible, shows connection status
Transcript panel	Last 5–10 lines of live conversation	Scrolls automatically, color-coded by speaker
Phase detector	Current meeting phase badge (e.g. Objection Handling)	Detected by LLM from transcript context
Suggestions panel	3 cards: Ask Next / Objection Response / Talking Point	Refreshes every ~15s or on trigger
Bottom bar	Pause · Docs · Settings buttons	Pause stops audio capture temporarily

4. Requirements

Functional Requirements

Capture mic + system audio simultaneously	Must have
Stream audio to transcription API (≤1s delay)	Must have
Display rolling transcript in UI	Must have
Upload sales docs at startup (PDF/TXT)	Must have
Retrieve relevant doc chunks via RAG	Must have
Generate 3 suggestions every ~15s or on trigger	Must have
Detect meeting phase (intro / discovery / objection / close)	Should have
Pause / resume listening button	Should have
Export transcript as .txt after meeting	Nice to have

Known Risks

HARD	System audio capture is OS-specific Windows uses WASAPI loopback. macOS requires BlackHole virtual audio driver (free). Linux uses PulseAudio. Separate code paths needed per OS.
------	---

MEDIUM	End-to-end latency Total latency from speech to suggestion can be 3–6s. Deepgram handles transcription fast; LLM adds 1–3s. Keep context small to minimize LLM response time.
LOW	PyInstaller packaging size PyQt5 + chromadb + numpy bundle can be 200MB+. Use --onedir mode (not --onefile) to avoid slow startup. Always test on a clean machine.

5. Build Plan — 5 Modules

Build each module independently, test it in your terminal, then wire them together. Do NOT start the next module until the current one works.

1	Audio Capture Record mic + system audio. Print raw audio data to terminal. Goal: see audio data flowing without silence. Tools: sounddevice, numpy, BlackHole (macOS) / WASAPI (Win) ~1 evening
2	Live Transcription Stream audio chunks to Deepgram via WebSocket. Print live transcript in terminal. Goal: your words appear as you speak. Tools: deepgram-sdk, websockets ~1 day
3	RAG Document Layer Load a sample sales PDF → embed → store in ChromaDB. Query with a sentence. Goal: get relevant doc chunks returned. Tools: chromadb, openai, pypdf2 ~1 day
4	LLM Suggestions Combine last 5 transcript lines + top 2 doc chunks → send to LLM. Get back 3 structured suggestions. Tools: anthropic / openai ~1 day
5	Desktop UI + Wiring Build PyQt5 window. Show transcript + suggestions panels. Wire all 4 modules together. Package with PyInstaller to .exe. Tools: PyQt5, pyinstaller, python-dotenv ~2 days

3-Weekend Timeline

Weekend 1	Audio capture working + live transcript in terminal	Module 1 + 2
Weekend 2	RAG layer with sample doc + LLM generating suggestions	Module 3 + 4
Weekend 3	PyQt5 UI + wire all modules + package to .exe + record demo	Module 5